CE/CS/SE 3354 · Software Engineering · Fall 2026

Project Milestone P2

Analysis and High-Level Design

Toulene

Team	Team 07, section 005
Document version	1.0 (baseline for P2; grows from your P1 version 0.8)
Date	Nov 7, 2026
Repository URL	https://github.com/sandip10nepal/cs3354-project-p1
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	Sandip Nepal

Team member	NetID	GitHub username
Sandip Nepal	SXN230108	sandip10nepal
Arpita Lamichane	DAL081920	arpita-a04
Mehak Jay	MXJ230030	mehakjay
Vasudha Silawat	VXS240036	vsilawat
Gabriel Suarez	GXS220049	Suarez04
Jarir Ali	JMA240001	Jarir-al
Manideep Pendyala	mcp230000	Manideep728

0 Revision History, Contribution Record and Changes Since P1

Version	Date	Author	Summary of change
0.8	Oct 18, 2026	Sandip Nepal (Integrator)	P1 baseline compiled and submitted.
0.8.1	Nov 6, 2026	Sandip Nepal	Added P2 repository scaffold, root README.md, and Section 0 / 0.2 P1 change control log (CH-01–CH-07).
0.8.2	Nov 6, 2026	Mehak Jay & Vasudha Silawat	Added /docs/elicitation/interviews.md, split QR-03/QR-05 (Sec 3.2), expanded UC-01 and UC-04 (Sec 4.2), and added QAS-01–03 (Sec 6.1).
0.8.3	Nov 7, 2026	Gabriel Suarez	Updated Sec 3.1 MoSCoW priorities & sources, expanded UC-05 (Sec 4.2), completed Sec 4.3 review log, and added UC-05 sequence diagram (Sec 5.2).
0.8.4	Nov 7, 2026	Jarir Ali	Added Domain Model (Sec 5.1), UC-01 sequence diagram (Sec 5.2), Architectural Style (Sec 6.2), Component View (Sec 6.3), and Deployment View (Sec 6.4).
0.8.5	Nov 7, 2026	Arpita Lamichane	Updated Sec 2.1 & 2.6, ran K1–K6 cross-checks (Sec 5.3), mapped objects & principles (Sec 6.5–6.6
0.8.6	Nov 8, 2026	Manideep Pendyala	Added ADRs & Trade-offs (Sec 7), committed architecture spike S-01 (Sec 8), and updated P3 Plan, Risks, AI log, and Open Questions (Sec 10–11).
1.0	Nov 8, 2026	Sandip Nepal (Integrator)	Integrated all merged team PRs, verified Section 0.1 commit/PR links, compiled final P2 baseline PDF into /docs/p2, and pushed Git tag p2.

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Sandip Nepal	Sec 0 (Revision History), Sec 0.1 (Contribution Record), Sec 0.2 (Changes Since P1 CH-01–CH-08), Sec 1 updates, Sec 10.4 (Repository layout), Sec 12 (Baseline v1.0), root README.md, and /docs/p2/ final PDF assembly.	657dae87(#7)	#8, #6
Arpita Lamichane	Sec 2.1 Context Diagram scope fix, Sec 2.6 Feasibility evidence update, Sec 3.5 pointer, Sec 5.3 Consistency Cross-Checks (K1–K6), Sec 6.5 Analysis-to-Architecture mapping, Sec 6.6 Design Principles, and Sec 9.1–9.2 Extended Traceability Matrix & Gap Analysis.		-
Mehak Jay	Stakeholder interview notes in /docs/elicitation/interviews.md ([R3]), Sec 2.2–2.3 updates, Sec 4.1 use-case selection table, and Sec 4.2 expanded UC-01 Reserve Table.	899db0f(#4)	#8
Vasudha Silawat	Sec 3.2 Quality Requirement split (QR-03 & QR-05), Sec 4.2 expanded UC-04 Manage Table Seating, and Sec 6.1 Quality-Attribute Scenarios (QAS-01–QAS-03).	4b6eaf8(#6)	#4

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Gabriel Suarez	Sec 3.1 MoSCoW priority distribution & validated sources, Sec 3.4 actor-goal table & diagram (docs/p2/diagrams/use-cases.drawio), Sec 4.2 expanded UC-05 Place Table Order, Sec 4.3 Self-Review Log, and Sec 5.2.2 UC-05 Scenario Table & Sequence Diagram (docs/p2/diagrams/seq-uc05.drawio).	d1c8ae7(#11)	-
Jarir Ali	Sec 5.1 Conceptual Domain Model & table, Sec 5.2.1 UC-01 Scenario Table & Sequence Diagram, Sec 6.2 Architectural Style & Alternatives, Sec 6.3 Component View, Sec 6.4 UML Deployment View, and ADR-01.	45be85d(#9)	#14
Manideep Pendyala	Sec 7.1 ADR-02 & ADR-03, Sec 7.2 Trade-off Summary, Sec 8 Prototype Evidence Spike S-01 (/spikes/S-01/), Sec 10.1–10.3 Roles, Plan to P3 & Risks (R1–R7), Sec 10.5 AI Declaration, and Sec 11 Open Questions (Q1–Q5).	23b0c64(#8)	#4

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	Added /docs/elicitation/interviews.md ([R3]) and updated Sec 2.2 & Sec 3.1 (FR-01 to FR-14) Source column from "assumption; pending validation" to cite [R2], [R3].	**P1 Feedback (C1):** Stakeholder needs rested on assumptions and [R3] pointed to /docs/elicitation/interviews.md, which was missing from the repository.	Validates FR-01–FR-14 against stakeholder interviews and closes Risk R1 in Sec 10.3.	Nov 5, 2026

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-02	Updated Sec 2.1 Context Diagram to remove out-of-scope "payment processing" and "third party delivery apps" arrows from External Services.	**P1 Feedback (C1):** Context diagram showed payment processing and third-party delivery apps that Sec 1.2 explicitly excluded from scope.	Aligns Sec 2.1 system boundary with Sec 1.2 Scope and Sec 6.3 Component View	Nov 6, 2026
CH-03	Updated Sec 2.6 Feasibility Summary with concrete evidence and corrected the Schedule row from "4-5 team members" to 7 team members.	**P1 Feedback (C1):** Feasibility verdicts were asserted rather than evidenced, and the schedule row misstated the team size.	Aligns Sec 2.6 Schedule feasibility with the 7-member P3 plan in Sec 10.2.	Nov 6, 2026
CH-04	Rebalanced Sec 3.1 (FR-01–FR-14) Priority column across the MoSCoW range: 9 MUST (FR-01–03, FR-06–07, FR-09–11, FR-13), 4 SHOULD (FR-04, FR-05, FR-08, FR-12), and 1 COULD (FR-14).	**P1 Feedback (C2):** All 14 functional requirements were marked MUST in P1; feedback asked to use the full MoSCoW range to show what could be cut.	Sets the P3 build order in Sec 10.2 and focuses full traceability chains in Sec 9.1 on MUST items.	Nov 6, 2026
CH-05	Split QR-03 in Sec 3.2 into QR-03 (99% uptime availability during operating hours) and QR-05 (100% retention of confirmed reservations and submitted orders across 5 restarts).	**P1 Feedback (C2):** QR-03 combined availability and data-retention into a single requirement.	QR-05 drives quality scenario QAS-03 (Sec 6.1) and ADR-03 (commit to PostgreSQL before notifying screens).	Nov 6, 2026
CH-06	Replaced duplicate expanded use cases in Sec 3.4 with a clean actor-goal summary table (UC-01–UC-08), moved turn-by-turn expansions (UC-01, UC-04, UC-05) to Sec 4.2, and replaced Sec 3.5 with a pointer to Sec 9.1.	**P2 Template & Feedback (C3):** P2 keeps Sec 3.4 at actor-goal level, expands 3+ use cases in Sec 4.2, and extends traceability in Sec 9.1.	Eliminates duplication and traces UC-01, UC-04, and UC-05 into Sec 5 and Sec 6.	Nov 7, 2026

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-07	Added root README.md explaining the repository layout (/docs/p1, /docs/p2, /docs/p2/diagrams, /docs/elicitation, /spikes) and updated Sec 10.1 & 10.4 with team communication frequency and PR rules.	**P1 Feedback (C4):** Main branch needed a README and /docs explanation, and the plan needed to state how and how often the team communicates.	Clarifies repo structure for grading and enforces peer-reviewed PRs.	Nov 6, 2026
CH-08	Adopted WebSocket server push (ADR-02) and commit-before-notify persistence (ADR-03) backed by Architecture Spike S-01 (30be6d7), and added new architectural risks R6 and R7 in Sec 10.3	**Spike Result (S-01):** Under degraded Wi-Fi, WebSocket push delivered 100% of orders within 5 s (p95 = 4082 ms), whereas 3 s polling delivered only 82% (p95 = 6188 ms), failing QAS-01.	Confirms ADR-01 and ADR-02, closes R2 and Q1/Q3, and raises R6, R7, Q4, and Q5 for P3.	Nov 7, 2026

1 to 3 Introduction, Overall Description and Requirements (carried forward)

1 Introduction

1.1 Purpose of this document and of the system

This document serves as the Phase 2 (P2) analysis and high-level design baseline requirements specification for Toulene, intended for course instructors, teaching assistants, and project stakeholders following ISO/IEC/IEEE 29148 standards. It outlines the initial system boundaries, constraints, stakeholders, and testable requirements to guide subsequent software design phases.

Toulene addresses the communication breakdown between restaurant front-of-house reservation desks and back-of-house kitchen staff, which currently relies on fragmented phone bookings, lost paper tickets, and uncoordinated table turnover during peak dining hours. The system provides a unified digital platform enabling customers to book tables online, hosts to manage real-time seating, and kitchen chefs to track, prioritize, and update meal preparation statuses via a digital Kitchen Display System (KDS).

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
Customer-facing web portal for browsing availability, selecting party sizes, and booking time slots.	Physical credit card payment processing hardware or PCI-DSS financial gateway integration (excluded to keep P1 scope focused on reservation and order workflow logic).
Host station dashboard for managing floor plans, assigning tables, and tracking walk-in queues.	Automated robotic food delivery or conveyor belts between the kitchen and dining tables (excluded as physically impractical for standard software engineering projects).
Kitchen Display System (KDS) interface for chefs to view, sort, and mark item preparation statuses.	Third-party food delivery app integrations like UberEats or DoorDash API synchronization (excluded as out of scope for baseline dine-in restaurant operations).
Automated text/email notifications alerting customers when their table is ready or food is prepared.	Inventory supply chain management and automated supplier reordering (excluded as the project is strictly bounded to reservation and order routing).

Why the exclusions were excluded: The exclusions focus on financial payment gateway compliance and third-party logistics integrations that exceed the software development boundaries and timeframe of a semester-long academic project.

1.3 Definitions, acronyms and abbreviations

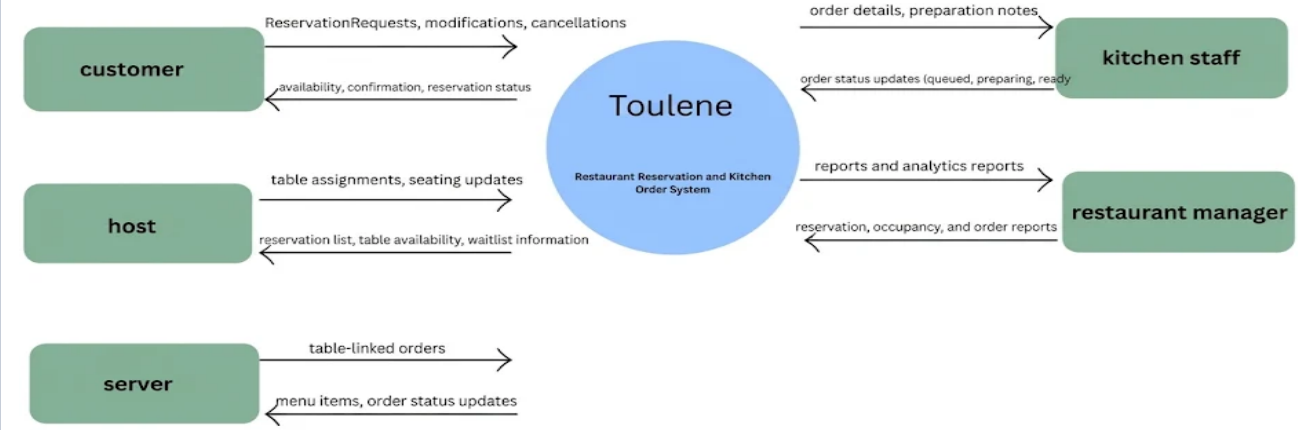
Term or abbreviation	Meaning as used in this project
KDS	Kitchen Display System; the digital screen interface used by back-of-house staff to manage incoming orders.
FOH	Front-of-House; staff members interacting directly with patrons, including hosts, servers, and managers.
BOH	Back-of-House; kitchen staff members responsible for food preparation and cooking.
Table Turn Time	The total duration a party occupies a table from seating to checkout and cleaning.

1.4 References

- **[R1]** ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.
- **[R2]** Standard Restaurant Hospitality Operations and Floor Management Guidelines, Rev. 2024.
- **[R3]** Stakeholder Interview Transcripts and Elicitation Notes, stored in repository /docs/elicitation/interviews.md.

2 Overall Description

2.1 Product perspective and context



2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
Customers	Reserve tables, fix reservations, and view reservation status.	Ability to view available time slots, make reservations online, receive confirmation notifications, change or cancel reservations, and check reservation status without calling the restaurant.	Researched reservation features used in systems like OpenTable and Yelp Reservations, combined with what we found from stakeholder interviews documented in /docs/elicitation/interviews. md.
Hosts	Manage table assignments and customer arrivals.	A real-time view of reservations, table availability, waitlists, customer check-ins, and tools to assign guests to tables quickly.	This comes from watching how seating workflows, and from what the hosts told us in their interviews /docs/elicitation/interviews. md).
Servers	View table status and customer orders	Access to assigned tables, customer orders, order	Researched Kitchen Display Systems (KDS) and how

Stakeholder	Role in relation to the system	What they need from it	How you learned this
		status, order updates, and communication with kitchen staff.	restaurant kitchens receive, prioritize, and track orders from our server interviews /docs/elicitation/interviews.md).
Kitchen Staff	Receive and track incoming food orders	Clear order tickets, updates when new orders arrive, information about dietary requests, order priority tracking, and notifications when orders are updated.	Reviewed features of restaurant management software and found some common operational needs like reservation tracking, occupancy monitoring, and performance reporting. Interviewed kitchen staff with kitchen staff documented in /docs/elicitation/interviews.md.
Restaurant Managers	Monitor reservations, restaurant occupancy, and order performance	Dashboards showing reservation activity, table occupancy, wait times, completed orders, peak business hours, and performance reports of the restaurant.	Research on restaurant management software and findings from restaurant manager interview documented in /docs/elicitation/interviews.md.

2.3 User characteristics

Customers: Restaurant guests use the system to reserve tables, check availability, modify reservations and view reservation status. Most customers can assume to know how to use a smartphone app or website, but they cannot be assumed to understand restaurant operations or technical system features. Customers can access the system from mobile phones, tablets, or computers. Since many customers may make reservations quickly, the system must be easy to use with clear instructions. Basic accessibility features: readable text, clear buttons, and simple layouts should be used.

Hosts: Restaurant hosts are responsible for managing reservations, assigning tables, and checking on arriving customers. They are expected to have basic computer skills and receive training on the system. They cannot be assumed to have advanced technical knowledge. Hosts usually use a desktop computer or tablet near the restaurant entrance. Because they work in busy environments and must get to customers quickly, the system should display information in real time and allow common tasks to be completed with limited steps.

Servers: Restaurant servers use the system to see table assignments, review customer orders, and check order status. Most servers can be assumed to have experience using touchscreen devices and restaurant technology,

but not advanced technical expertise. Servers will likely use tablets or restaurant terminals during their shifts. Since servers are frequently moving between tables and maybe handling multiple customers at once, the system should allow fast access to information to prevent unnecessary navigation.

Kitchen Staff: Restaurant kitchen staff use the system to receive, track, and update food orders. They can be assumed to understand restaurant order workflows but may have different levels of computer experience. Kitchen staff usually use a kitchen display screen or terminal. Because the kitchen environment is fast paced and time sensitive, order information must be presented clearly, accurately, and in a format that is easy to read. The system should reduce clutter and make important updates the priority.

Restaurant Managers: Restaurant managers use the system to monitor reservations, occupancy levels, order activity, and restaurant performance. Managers are expected to have moderate experience with business software and reporting tools. They could access the system through desktop computers, tablets, or laptops. Managers often need to make operational decisions quickly, so the system should provide organized reports, dashboards, and summary information that is easy to understand.

2.4 Constraints

Platform	Toulene is web-based. The customer portal must run in current versions of Chrome, Safari, Firefox, and Edge on phones, tablets, and computers without installing anything.
Kitchen hardware	KDS runs on kitchen screen, terminal or tablet in a fresh environment. So, the interface must use large touch targets and texts that is easy to read at a glance
Network	The restaurant's local wifi and internet connection in the only link between front of house and backhouse so toluene depends on it
privacy	Customer contact details must be handled under applicable privacy laws and must not be shared with third parties without the customer's consent.
Payments	Payment processing is out of scope, so Toluene must not process payments or collect or store card numbers.
Accessibility	The customer-facing interface must meet WCAG 2.1 Level AA, including readable text, clear buttons, and simple layouts.
Project	Development must be completed within the CS 3354 semester schedule by a team of seven students with no budget, so only free or open-source tools can be used.
Operations	Toluene must keep running through peak dining hours without staff having to restart it.

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	The restaurant has a stable internet and Wi-Fi connection during service.	Reservations and orders cannot sync, and the KDS stops updating, so kitchen staff miss new orders.
A2	Hosts and servers are trained and enter all reservations and orders through Toulene.	Orders taken outside the system never reach the kitchen, and the host's table view becomes inaccurate.
A3	Customers give a valid phone number or email when booking.	Confirmations and table-ready notifications cannot be sent, which increases no-shows.
#	Assumption or dependency	What breaks if it does not hold

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Feasible	For the restaurant features that we have such as order tracking, dashboards, and reservations, we will use the technologies Node.js, and React. We can implement all of these basic functionalities with the web tools we use and our skills as a team.
Operational	Feasible	The evidence is that our system aligns with real workflows of restaurants and needs of the people that use them such as servers, customers, hosts, managers, and kitchen staff. Our interfaces are made well for restaurant systems, and they are very simple to use for training as well.
Schedule	Feasible	Feasible because our scope is basically limited to our reservation and our workflow logic. We have 14 functional requirements and 8 main use cases. These requirements and use cases are most definitely achievable with the semester and 7 team members.
Economic / resource	Feasible	Feasible because we can build this with tools that are readily available and that are either free or barely cost anything. We will be using university resources and software that is open sourced.

3 Specific Requirements

3.1 Functional requirements

ID	Requirement	Priority	Source	Acceptance criterion
FR-01	The system shall display available reservation time slots for a customer's selected date and party size based on table capacity and existing reservations.	MUST	Customer need [R2], [R3]	Given a table for four is available at 6:00 PM, a search for four guests includes that slot; a search for five excludes it unless another suitable table is available.
FR-02	The system shall create a reservation with a unique confirmation number when a customer submits valid contact details, party size, and an available time slot.	MUST	Customer need [R2], [R3]	Submitting valid details for an available slot creates one reservation and displays its unique confirmation number.
FR-03	The system shall prevent reservations that allocate the same table to overlapping reservation periods.	MUST	Restaurant staff need [R2], [R3]	When two customers attempt to reserve the same table for overlapping periods, only one reservation is confirmed.
FR-04	The system shall allow a customer to modify their upcoming reservation to an available date, time, or party size.	SHOULD	Customer need [R2], [R3]	A valid change updates the reservation and releases its previous allocation; an unavailable change preserves the original reservation.
FR-05	The system shall allow a customer to cancel their upcoming reservation.	SHOULD	Customer need [R2], [R3]	Canceling a confirmed reservation changes its status to canceled and makes its allocated table available for that period.
FR-06	The system shall allow restaurant staff to mark a confirmed reservation as seated and associate it with the occupied table.	MUST	Host need [R2], [R3]	Selecting a confirmed reservation and an available table records the table assignment and changes the reservation status to seated.
FR-07	The system shall allow a server to create a table-linked order containing available menu items, quantities, and optional preparation notes.	MUST	Server need [R2], [R3]	A server adds two available items with quantities and notes, saves the order, and retrieves it under the correct table.
FR-08	The system shall reject an order submission containing an item currently marked unavailable.	SHOULD	Kitchen staff need [R2], [R3]	Submitting an order with an unavailable item identifies the item and leaves the order unsubmitted.
FR-09	The system shall transmit a submitted order to the kitchen queue with its order identifier, table number, items, quantities, preparation notes, and submission time.	MUST	Server and kitchen staff need [R2], [R3]	Submitting a valid order produces a kitchen queue entry whose details match the submitted order.
FR-10	The system shall allow kitchen staff to change an order's status from queued to preparing and from preparing to ready.	MUST	Kitchen staff need [R2], [R3]	Kitchen staff advance a queued order through preparing to ready, and each saved status appears on the order.
FR-11	The system shall display the current kitchen order status to the server responsible for the table.	MUST	Server need [R2], [R3]	After kitchen staff mark an order ready, the responsible server's order view displays ready.
FR-12	The system shall allow a server to mark a ready order as served.	SHOULD	Server need [R2], [R3]	Marking a ready order as served updates its status to served and removes it from the active kitchen queue.

ID	Requirement	Priority	Source	Acceptance criterion
FR-13	The system shall allow restaurant staff to mark a table as available after its dining party leaves.	MUST	Restaurant staff need [R2], [R3]	Releasing an occupied table changes its current occupancy status to available while preserving its future reservations.
FR-14	The system shall allow authorized restaurant staff to mark a menu item as available or unavailable for ordering.	COULD	Restaurant staff need [R2], [R3]	Marking an item unavailable prevents it from being added to a new order; restoring availability permits it again.

Priority distribution: 9 MUST (64.3%), 4 SHOULD (28.6%), 1 COULD (7.1%). Core reservation, seating, order submission, kitchen preparation, and order tracking remain MUST. No requirement is withdrawn.

3.2 Quality requirements

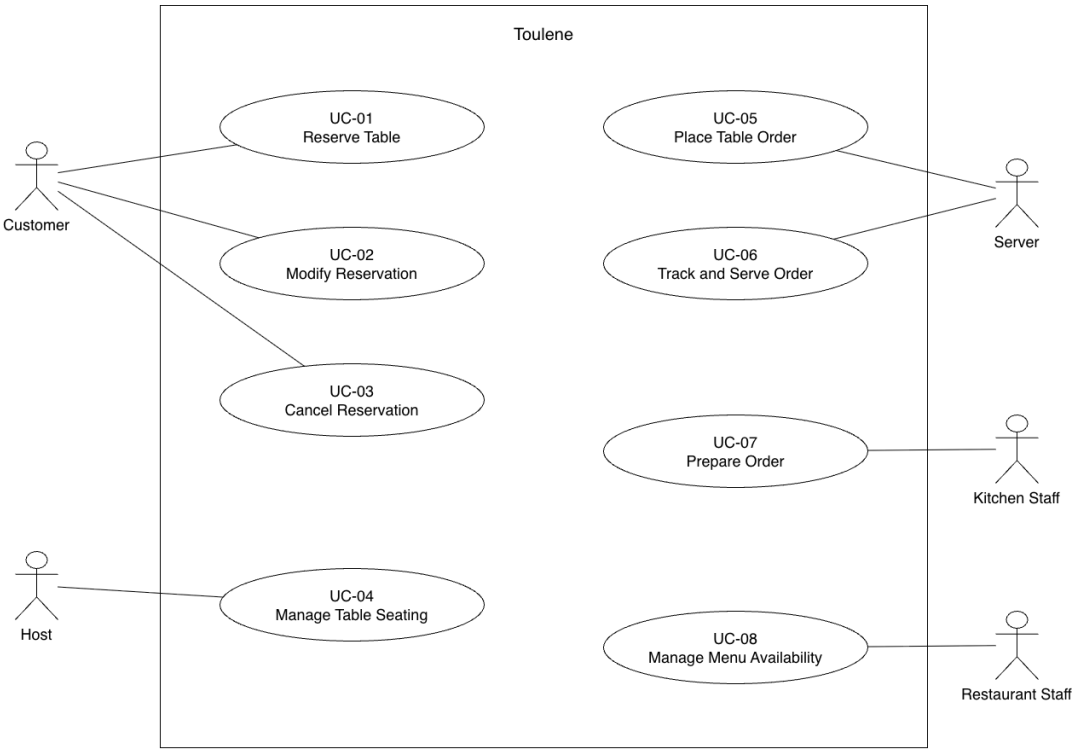
ID	Requirement — with threshold, condition and method	Priority	Source	How it will be verified
QR-01	Toulene shall display a submitted order on the KDS within 5 seconds for 95% of orders, with 20 active tables on the restaurant Wi-Fi.	MUST	Kitchen staff need	Submit 50 test orders and compare submission and display timestamps.
QR-02	Toulene shall load any customer-facing page within 3 seconds for 95% of requests with 100 concurrent users.	MUST	Customer need	Load test with 100 simulated users.
QR-03	Toulene shall be available at least 99% of the time during operating hours.	MUST	Restaurant manager need	Monitor uptime during a multi-day test over simulated operating hours; availability = operating time available / total operating time × 100.
QR-04	Kitchen staff shall change an order’s status in 2 taps or fewer, using touch targets of at least 44x44 pixels.	SHOULD	Kitchen staff need	Task walkthrough with kitchen-role testers and a design inspection of the KDS.
QR-05	Toulene shall retain 100% of confirmed reservations and submitted orders across a system restart.	MUST	Restaurant manager need	Create 20 confirmed reservations and 20 submitted orders; restart the system 5 times and verify that all 40 records remain intact after every restart.

3.3 Constraint requirements

CON-01 and CON-02 are unchanged from the P1 v0.8 baseline.

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	Toulene shall run in the latest two versions of Chrome, Safari, Firefox, and Edge with no installation.	customers use their own devices	Customers book on their own phones and computers, and the restaurant will not install software for guests.
CON-02	The KDS shall be fully usable by touch alone on a screen of at least 10 inches.	Kitchen staff	Cooks work with gloved hands and have no keyboard or mouse at the station

3.4 Use cases



Editable source: docs/p2/diagrams/use-cases.drawio. The actor-goal list remains unchanged; detailed expansions belong in Section 4.2.

ID	Actor goal	Primary actor
UC-01	Reserve Table	Customer
UC-02	Modify Reservation	Customer
UC-03	Cancel Reservation	Customer
UC-04	Manage Table Seating	Host
UC-05	Place Table Order	Server
UC-06	Track and Serve Order	Server

ID	Actor goal	Primary actor
UC-07	Prepare Order	Kitchen Staff
UC-08	Manage Menu Availability	Authorized Restaurant Staff

3.5 Requirements to use case traceability

Requirements-to-use-case traceability has been extended in Milestone P2 to cover analysis classes, architectural components, quality-attribute scenarios, and ADRs; see Section 9.1 (Extended Traceability Matrix) and Section 9.2 (Gap Analysis).

4 Use-Case Analysis

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-01	Reserve Table	This use case was selected because a Reserve Table is the first step a customer takes when booking a table, and the rest of the reservation system builds on the reservations this use case creates. This use case covers FR-01, FR-02, and FR-03 and deals with checking which tables are free, checking the reservation details, picking a table that fits party sizes, and stopping two parties from getting booked for the same table at the same time. Without this use case working properly, the host has no reservations to seat people from, and since booking is the first step for the customer, a bad booking flow makes the whole visit start off wrong. The availability checking and conflict prevention matter because if two parties end up booked for the same table at the same time, the host must turn one of them away at the door.	Checking what tables are free, checking the reservation details, picking a table that fits, stopping double bookings, and confirming the reservation.
UC-05	Place Table Order	This use case was selected because Place Table Order connects the servers taking orders to the kitchen staff making them, so if this use case breaks down the kitchen gets the wrong orders or gets them late. This use case affects how accurate the orders are and how fast the restaurant runs, since a wrong item or a wrong quantity means the kitchen has to remake food and the table waits longer. It also deals with checking the order before it goes to the kitchen, marking that an order is unavailable (lack of ingredients, etc), and keeping the server and the kitchen in sync on what was ordered and what still needs to go out.	Checking what is on the menu, checking the order, dealing with items that are out, saving the order, and sending it to the kitchen queue.

Use case	Name	Why it was selected	Non-trivial steps
UC-04	Manage Table Seating	This use case was selected because Manage Table Seating links the reservations customers made and the host seating people at the door, so this use case decides how well the tables get used across a night. If the host seats parties badly, tables sit empty while people wait, and if the host seats parties well, more parties get served. Manage Table Seating also holds a lot of the business rules for giving out tables and freeing them up again, like which tables fit which party sizes, when a table counts as taken, and when a table opens back up after a party leaves.	Checking if the party arrived, finding their reservation, giving them a table, keeping track of which tables are taken, updating the seating status, and freeing the table when they leave.

4.2 Expanded use cases

UC-01 [Reserve Table]

Primary actor	Customer
Supporting actors and external systems	None
TUCBW (this use case begins when)	This use case begins when a customer wants to book a table for a certain date and time.
TUCEW (this use case ends when)	This use case ends when the customer gets a confirmation that their reservation went through.
Preconditions	- The restaurant's rules, hours, and table size are already set up in the system - A table is free at the time the customer wants.
Postconditions	- A reservation is saved for a set date and time. - The table fits the party size. - Booking has it's own confirmation number - The table is blocked off for overlapping times
Traces to requirements	FR-01, FR-02, FR-03, QR-02, QR-03, CON-01

Actor input / action	System response
	(0) The system requests the reservation information that is needed to check if there is availability.
(1) The customer provides information about the date, time, and party-size for their reservation.	(2) The system finds reservation options using the customers information.
(3) The customer selects the reservation option they want.	(4) The system requests contact information needed for completing the reservation.

Actor input / action	System response
(5) The customer provides the contact information and confirms the reservation.	(6) The system creates the reservation and hands back a confirmation number.

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	(2)	There is no table free that matches the date, time, and party size the customer asked for.	The system tells the customer there is no table available that matches the requested date, time, and party size. And asks them to pick a different date, time, or party size	(1)
A2	(5)	The customer left out some of the required information to complete the reservation, or they typed in something wrong.	The system points out what needs to be fixed and asks the customers to fill the information in again.	(5)

UC-05 [Place Table Order]

Field	Content
Primary actor	Server
Supporting actors and external systems	None; kitchen preparation belongs to UC-07.
TUCBW	The Server needs to place an order for a dining party.
TUCEW	The Server receives submission confirmation, cancellation confirmation, or a failure message.
Preconditions	The Server is authorized. Table and menu information exists.
Success postconditions	One table-linked Order appears once in KitchenQueue with matching details and queued status. The Server receives confirmation.
Minimal guarantee	A rejected or cancelled attempt creates no submitted Order or kitchen entry.
Traces to requirements	FR-07, FR-08, FR-09; QR-01 (kitchen delivery time).

Main success scenario

Actor input / action	System response
—	(0) Lists occupied Tables by table number for a new Order.
(1) The Server chooses a Table and requests a new Order.	—
—	(2) Lists MenuItems and their availability.
(3) The Server gives MenuItems, quantities, and optional preparation notes.	—

Actor input / action	System response
—	(4) Shows the draft: table number, MenuItems, quantities, and notes. Requests confirmation or changes.
(5) The Server confirms the Order.	—
—	(6) Confirms kitchen submission with order number, table number, items, quantities, notes, submission time, and queued status.

Alternate flows

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	(0)	No occupied Table exists.	The system reports no occupied Table. Seating must be completed through UC-04.	Ends; restart at (0) after seating.
A2	(2) or (6)	The chosen Table is no longer occupied.	The system rejects the Table. The Server chooses another or cancels.	Rejoins (2) after a new (1), or ends through A6.
A3	(4)	Unavailable item, empty Order, or invalid quantity (not a positive whole number).	The system identifies the problem and leaves the Order unsubmitted. The Server corrects it or cancels.	Rejoins (4) after a corrected (3), or ends through A6.
A4	(5)	The Server finds an error in the summary.	The Server revises the items, quantities, or notes. The system shows the updated draft.	Rejoins (4), then (5).
A5	(6)	A MenuItem became unavailable after the summary was presented.	The system identifies the item and reports no submission. The Server revises or cancels.	Rejoins (4) after a corrected (3), or ends through A6.
A6	(1), (3), or (5)	The Server abandons the attempt before submission.	The Server cancels. The system confirms that no Order was submitted.	Ends without submission.
A7	(6)	The Order cannot be accepted for kitchen preparation.	The system reports failure: no submitted Order or kitchen entry. The Server retries or cancels.	Rejoins (5) for a new confirmation, or ends through A6.

UC-04 [Manage Table Seating]

Primary actor	Host
Supporting actors and external systems	None.
TUCBW (this use case begins when)	An arriving party with a confirmed reservation asks to be seated.
TUCEW (this use case ends when)	The host receives confirmation that the party is seated; later, after the party leaves, the host receives confirmation that the table is available.
Preconditions	Tables and capacities are defined. A confirmed reservation exists for seating; an occupied table exists for release.
Postconditions	The reservation is marked seated and linked to one occupied table. After release, that table is available and its future reservations are preserved.
Traces to requirements	FR-06, FR-13. No direct QR or CON trace.

Actor input / action	System response
	(0) The system presents confirmed reservations due now and their allocated tables and current occupancy.
(1) The host identifies the arriving party's reservation.	(2) The system confirms the reservation and gives the host its party size, time, and allocated table.
(3) The host confirms arrival and asks to seat the party.	(4) The system checks that the allocated table is available for this party and confirms the proposed assignment.
(5) The host accepts the proposed table.	(6) The system marks the reservation seated and the table occupied, then confirms the assignment to the host.
(7) After the party leaves and the table is cleared, the host asks to release that table.	(8) The system identifies the occupied table and its current party, then asks the host to confirm the release.
(9) The host confirms that the party has left and the table is clear.	(10) The system marks the table available, preserves future reservations, and confirms the release to the host.

Alternate flow	At step	Condition	What happens	Ends or rejoins at

A1	(2)	No confirmed reservation is found, or it was marked no-show.	The system tells the host that the party cannot be seated under that reservation. The host may help the party start a new reservation (UC-01).	Ends.
A2	(4)	The allocated table is unavailable.	The system proposes the smallest available table that fits the party and does not conflict with another reservation.	Rejoins at (5); if none exists, A3.
A3	(4)	No suitable table is available.	The system tells the host that seating cannot yet be confirmed. The host tells the party the expected wait.	Rejoins at (3) when a table becomes available, or ends if the party leaves.
A4	(5)	The host chooses another suitable table.	The system checks that table's availability and reservation conflicts; it accepts the choice or explains why it cannot be used.	Rejoins at (6) if accepted, otherwise (5).
A5	(6)	The assignment cannot be saved.	The system reports failure and keeps the reservation confirmed and the table available; no partial seating is recorded.	Rejoins at (5) if the host retries, otherwise ends.
A6	(8)	The selected table is not occupied.	The system tells the host there is nothing to release; no table or reservation changes.	Ends.
A7	(10)	The release cannot be saved.	The system reports failure and leaves the table occupied; the host can retry after checking its status.	Rejoins at (7) if retried, otherwise ends.

4.3 Self-review log

Criteria: Lesson 13 rules in the P2 template. Drafts: UC-01 PR #4 (cd6a5a7), UC-04 PR #5 (68b8da3), and current UC-05.

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-05	Step (0) and separate turns	Missing initial information and two-column turns.	Added Step (0) and Actor/System turns.

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-05	Visible ending, alternate flows, and no background steps	Unclear failure/cancellation endings; internal recording mixed with actor steps.	Added TUCEW and A1–A7; moved internal messages to 5.2.2.
UC-01	Step (0), turns, and visible ending	Single list instead of two columns; (2) finds options without presenting them.	Open: use two columns; present options at (2).
UC-01	Precondition or alternate flow	Free-table precondition conflicts with A1; no last-minute availability flow.	Open: remove that precondition; add an availability recheck/failure flow.
UC-01	Clear alternate-flow turns	A2 starts at actor step (5), without a system validation response.	Open: validate at (6), explain corrections, return to (5).
UC-04	Step (0) and separate turns	Step (0) is a trigger; single list includes actions outside Host/System turns.	Open: add initial information and two-column Host/System turns.
UC-04	TUCBW, TUCEW, and visible ending	Missing TUCBW/TUCEW; (7)/(11) lack visible confirmation.	Open: add fields and seating/release confirmations, each with a trigger and ending.
UC-04	Preconditions and alternate-flow assumptions	Reservation precondition conflicts with 3a; seating rules need a source.	Open: handle reservation lookup as a flow; confirm seating rules.

UC-01/UC-04 have no UI or implementation terms. Open fixes remain with their owners. This log is not a submitted GitHub review. Domain checks await Section 5.1.

5 Analysis Models

Chapter 7, second half. What has to exist inside the system for the use cases in section 4 to happen, and how those parts talk to each other.

5.1 Domain model

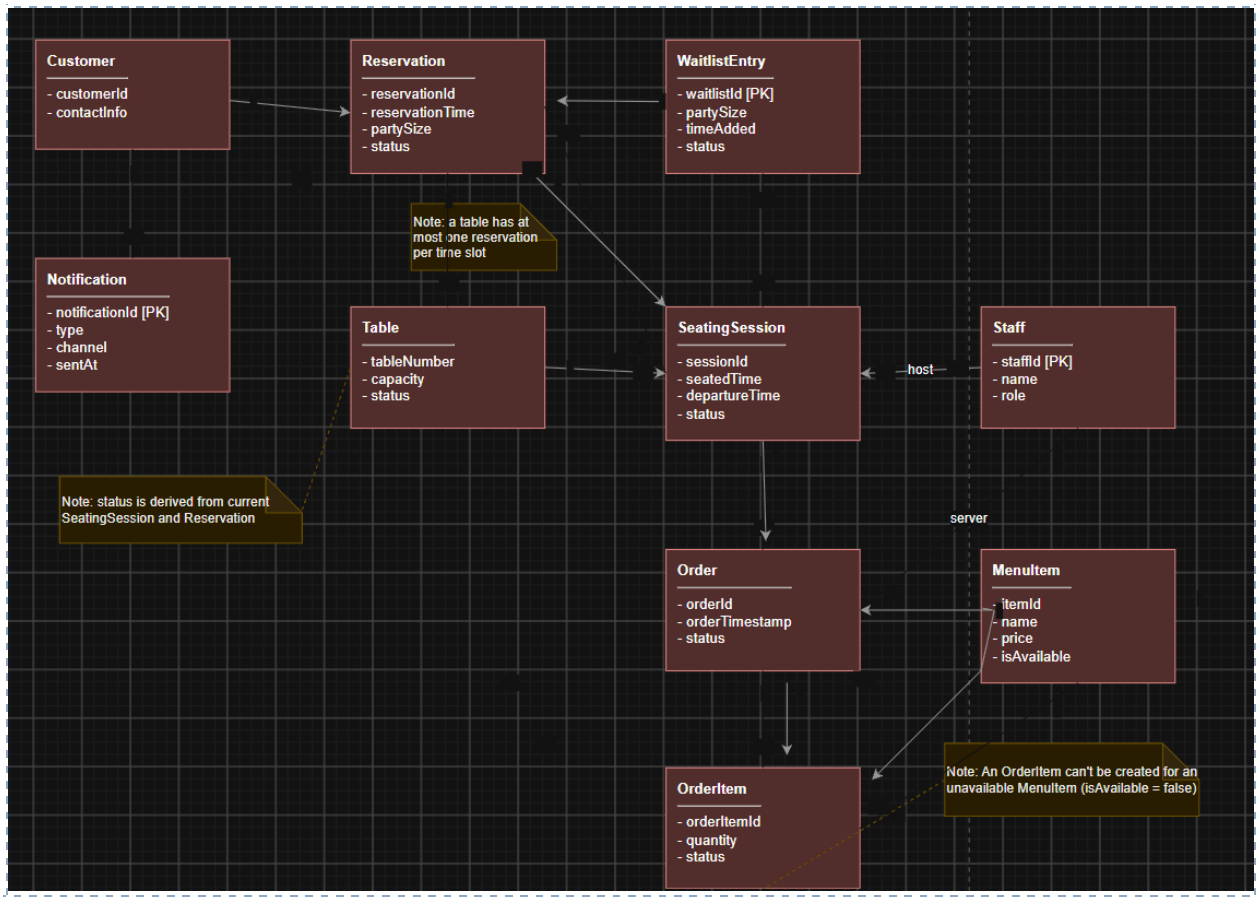


Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Customer	A person who books a table at the restaurant	customerId, contactInfo	Customer 1-0 Reservation Customer 1-0 Notifcation
Reservation	A customer's request to hold a table for part at a specic time	reservationId, reservationTime, partySize, status	Reservation * - 1 customer Reservation * - 1 table Reservation 0..1 – 1 seatingSession

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
SeatingSession	One party's time from beginning of seating to end of seating	sessionId, seatedTime, departureTime, status	SeatingSession - 1 table; SeatingSession 0..1 waitlistentry; seatingSession 0..1 – 1 SeatingSession 1-0..* Order
Table	A physical table with a fixed capacity	tableNumber, capacity	Table 1 – 0..* Reservation; Table 1 -0..* SeatingSession
WaitlistEntry	A walk in party waiting for a table	waitlistId, partySize, timeAdded, status	WaitListentry 0..1 – 1 Reservation
Staff	A host, server, or kitch worker who uses system	staffId, name, role	Staff 1 — 0..* SeatingSession (host who seated it)
Order	A set of items a server sends to the kitchen seating for a seating	orderId, orderTimeStamp, status (pending, preparing, ready, served)	Order * — 1 SeatingSession; Order 1 — 1..* OrderItem;
Orderitem	One menu item with an order	orderItemId, quantity, specialRequests, status	OrderItem * — 1 Order; OrderItem * — 1 MenuItem
MenuItem	A dish or drink from the restaurant menu	itemId, name, price, isAvailable	MenuItem 1 — 0..* OrderItem
Notification	A message sent to a customer about their table or food	Notifcationid, type, sentAt, channel (phone, email, etc)	Customer

5.2 Object interaction models

5.2.1 UC-01 Reserve Table

(a) Non-trivial steps. List the steps from 4.2 that need objects inside the system, and say in one line why each of the others is trivial.

UC – 0 [Reserve Table]1:

Step (2): Requires checking table availability and capacity against partySize and searching existing reservation records to prevent overlapping double bookings.

Step (6): Requires creating and saving a persistent reservation linked to a table, gerneating unique keys and updating booking statuses, supposedly.

Why Trivial:

Step 0: Trivial because it only renders an empty booking input form without touching domain data.

Step 1: Trivial because it is raw user UI data entry prior to domain validation

Step 3: Trivial because it is a user selection action on the interface picking from already getten (using getMethods) options.

Step 4: Trivial because it just displays a prompt or form fields on the client UI requesting customer contact details

Step 5: Trivial because it captures the contact strings and receives a user button click

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
0	System	Requests Reservation parameters	Input form UI	Customer
1	Customer	Submit target booking details	Date, time, party size	Reservation screne/interface
2	System	Searches matching table options	Requested party size, schedule	[table], [Reservation]
3	Customer	Selects preferred booking slot	Available table options	Reservation Interface
4	System	Prompts for contact info	Contact input form	Customer
5	Customer	Enters contact data	Customer name, phone, email	Reservation Interface
6	System	Creates reservation and returns confirmation number to user	Selected table slot, customer details	Customer, Reservation, Table

(c) Analysis sequence diagram.

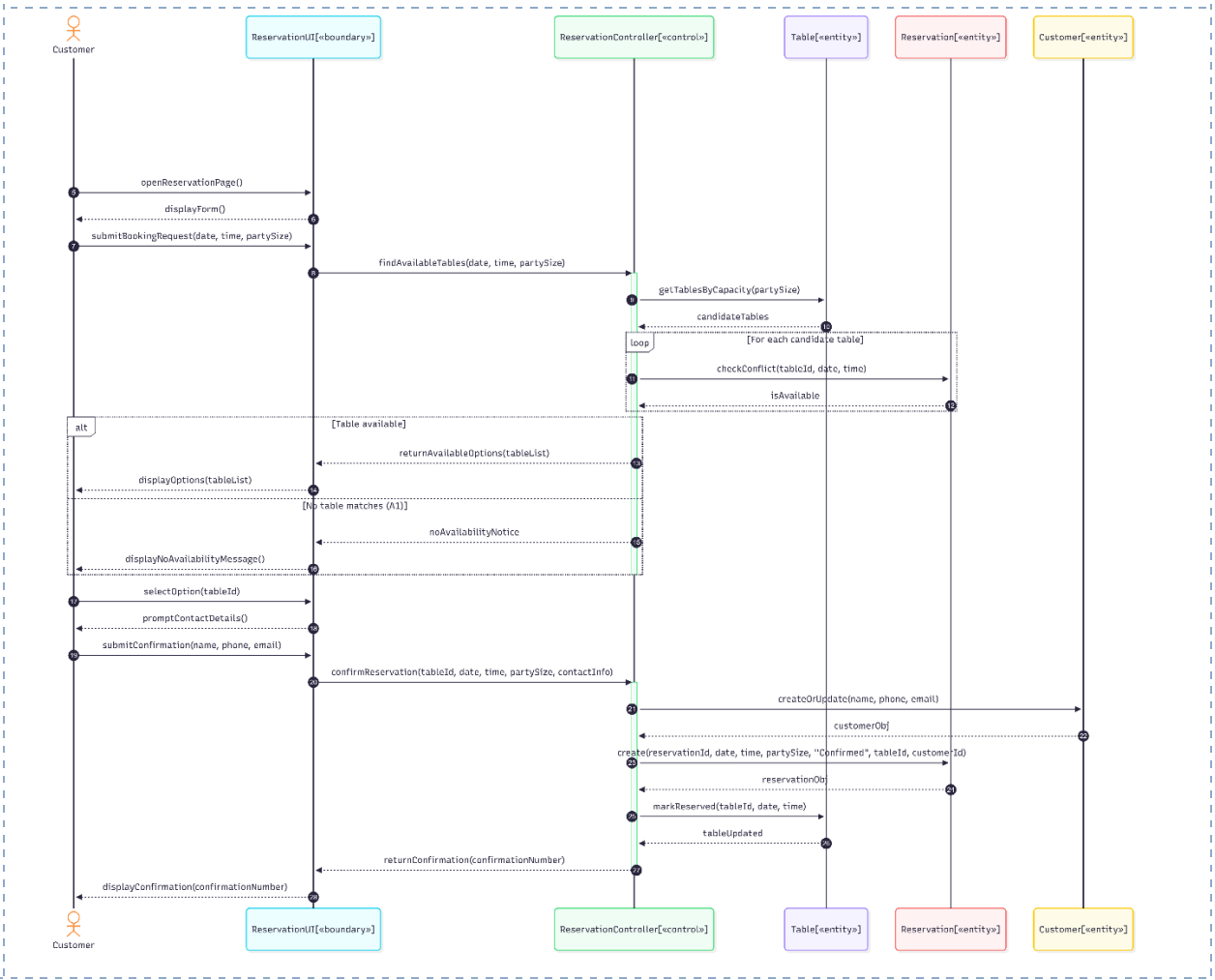


Figure 5.2.1 Analysis sequence diagram for UC-01; message numbers match the scenario table

5.2.2 UC-05 Place Table Order

(a) Non-trivial steps

Section 4.2 step	Why the step needs objects
(0)	Table occupancy determines which table numbers can be offered.
(2)	MenuItem availability must be obtained for the chosen Table.
(4)	Order composes OrderLines and supplies a complete, table-linked draft for review.
(6)	Table occupancy and MenuItem availability must be rechecked, the Order finalized, and kitchen acceptance established before acknowledgment.

Steps (1), (3), and (5) are simple inputs: Table choice, Order details, and confirmation. The table and diagram cover the main success flow.

Objects: OrderPlacement is the declared interaction controller for UC-05, and Server is the external primary actor. Domain classes participating in this interaction
are Table, MenuItem, Order, OrderLine (representing OrderItem in Section 5.1),
and KitchenQueue (representing the active queued view of Order / OrderItem instances filtered by status = queued in Section 5.1).

(b) Scenario table

M01–M21 match the diagram; parentheses identify Section 4.2 steps.

Step	Subject	Subject action	Other data / objects	Object acted upon
M01 (0)	OrderPlacement	asks for occupied tables	table occupancy	Table
M02 (0)	Table	provides occupied table numbers	table numbers	OrderPlacement
M03 (0)	OrderPlacement	presents occupied tables	table numbers	Server
M04 (1)	Server	identifies table and requests order	table number	OrderPlacement
M05 (2)	OrderPlacement	asks for menu availability	selected Table	MenuItem
M06 (2)	MenuItem	provides items and availability	item names, availability	OrderPlacement
M07 (2)	OrderPlacement	presents items and availability	item names, availability	Server
M08 (3)	Server	supplies order contents	MenuItems, quantities, preparation notes	OrderPlacement
M09 (4)	OrderPlacement	composes draft order	Table, OrderLines with MenuItems, quantities, preparation notes	Order
M10 (4)	Order	provides draft summary	table number, item names, quantities, preparation notes	OrderPlacement
M11 (4)	OrderPlacement	presents draft for confirmation	complete draft summary	Server
M12 (5)	Server	confirms draft order	reviewed draft	OrderPlacement
M13 (6)	OrderPlacement	rechecks table occupancy	chosen Table	Table
M14 (6)	Table	confirms table is occupied	table number, occupancy	OrderPlacement
M15 (6)	OrderPlacement	rechecks item availability	all MenuItems in Order	MenuItem
M16 (6)	MenuItem	confirms all items available	current availability	OrderPlacement
M17 (6)	OrderPlacement	finalizes submitted order	reviewed draft, unique order number, submission time, queued status	Order

Step	Subject	Subject action	Other data / objects	Object acted upon
M18 (6)	Order	provides submitted order details	order number, Table, OrderLines, submission time, queued status	OrderPlacement
M19 (6)	OrderPlacement	places order in kitchen queue	submitted Order and matching details	KitchenQueue
M20 (6)	KitchenQueue	acknowledges one queued order	order number, queued status	OrderPlacement
M21 (6)	OrderPlacement	acknowledges submitted order	order number, table number, items, quantities, notes, submission time, queued status	Server

A7 requires that failed kitchen acceptance leave no submitted Order or kitchen entry.

(c) Analysis sequence diagram

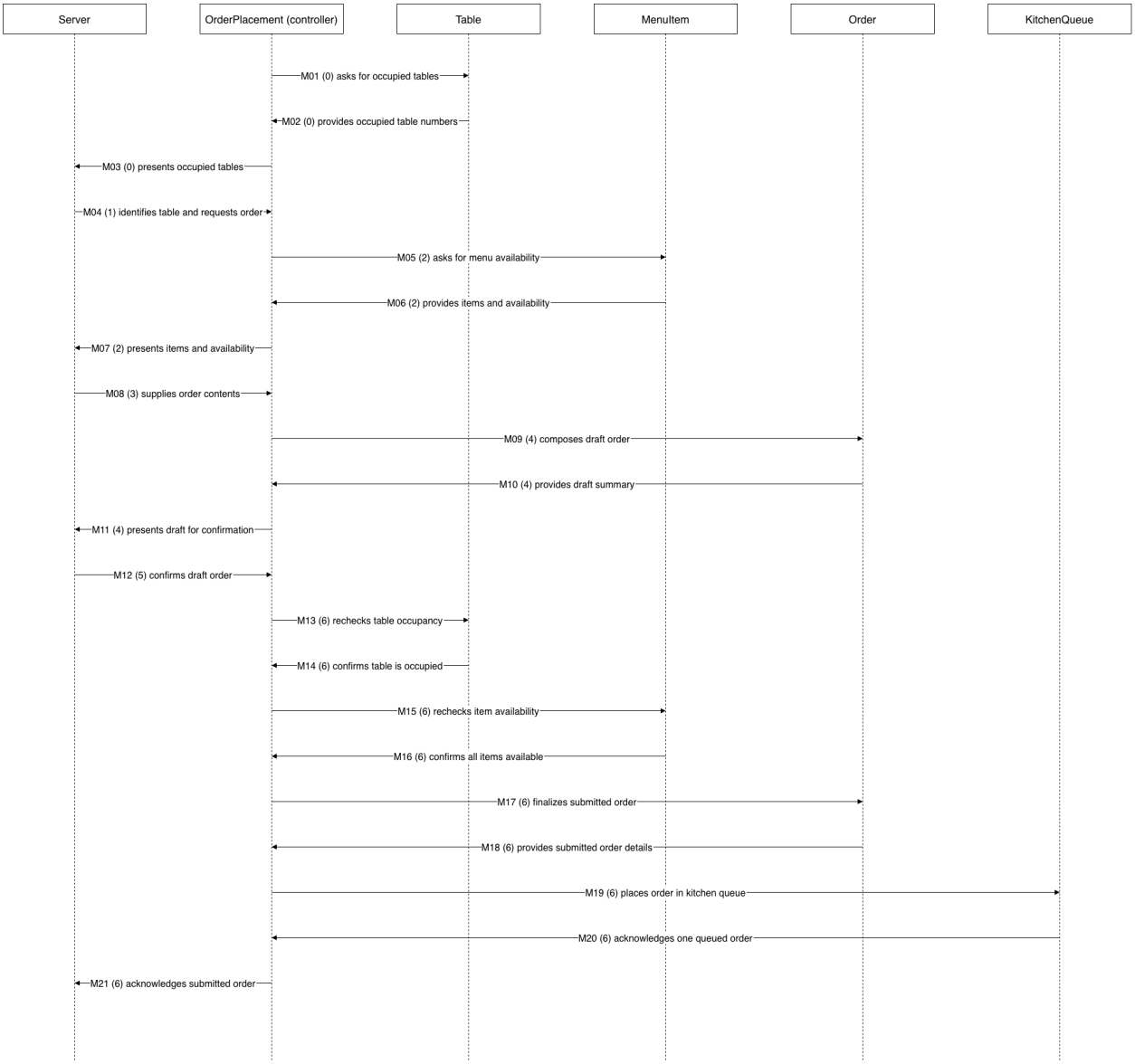


Figure 5.2.2. UC-05 main success flow. Source: docs/p2/diagrams/seq-uc05.drawio.

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	**Pass.** Every actor input in UC-01, UC-04, and UC-05 maps to domain attributes in Section 5.1: reservation date/time (Reservation.reservationTime), party size (Reservation.partySize), customer contact details (Customer.contactInfo), table selection (Table.tableNumber, Table.capacity), seating status (SeatingSession.status, Reservation.status), and order details (MenuItem.itemId, OrderItem.quantity, OrderItem.specialRequests).	Sec 4.2 & Sec 5.1
K2 Every object on a sequence diagram is a domain class or a declared interaction object	**Pass.** Initially, UC-05 used OrderLine and KitchenQueue while Section 5.1 used OrderItem and Order status. Resolved by declaring ReservationUI and ReservationController (UC-01) and OrderPlacement (UC-05) as interaction controllers/boundaries, and explicitly mapping OrderLine to OrderItem and KitchenQueue to queued Order records.	Sec 5.1, 5.2.1, 5.2.2 & 6.5

Check	Result (pass, or what failed)	Fixed in
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	**Pass.** In UC-05 (5.2.2), messages M01–M21 match the 21 scenario table rows 1-to-1. In UC-01 (5.2.1), the sequence interactions map directly to steps (0)–(6) in Scenario Table 5.2.1(b).	Sec 5.2.1 & Sec 5.2.2
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	**Pass.** UC-01 non-trivial steps (2) (availability search and conflict check) and (6) (reservation creation and table assignment) are realized in Table 5.2.1(b) rows 2 and 6. UC-05 non-trivial steps (0), (2), (4), and (6) are realized across rows M01–M03, M05–M07, M09–	Sec 5.2.1 & Sec 5.2.2
K5 Every analyzed use case traces to at least one functional requirement	**Pass.** UC-01 traces to FR-01, FR-02, and FR-03; UC-04 traces to FR-06 and FR-13; and UC-05 traces to FR-07, FR-08, and FR-09.	Sec 3.5, 4.2 & 9.1

Check	Result (pass, or what failed)	Fixed in
K6 The last messages of each diagram make the use case postcondition true	**Pass.** UC-01 ends by persisting the confirmed Reservation, updating Table allocation, and returning the unique confirmation number to the Customer. UC-05 ends at M17–M21 by finalizing the Order, queuing it in KitchenQueue with queued status, and returning confirmation to the Server	Sec 5.2.1 & Sec 5.2.2

6 High-Level Design (Architecture)

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance (Latency)	A server submits an order while 20 tables operate simultaneously on wifi	Kitchen Display System and Order Service	Ingests, persists, and renders the incoming order on the kitchen display	Order appears on display in under or at 5 seconds for 95% or more orders	QR-01
QAS-02	Performance (Throughput / Latency)	100 Concurrent customers browse and navigate booking pages during peak traffic hours	Customer web application and Reservation Service	Process requests and renders complete page content to client browsers	Page loads in under 3 seconds for more than 95% of navigations	QR-02
QAS-03	Availability/ Fault Tolerance	Primary server or host environment undergoes a crash or planned restart during operating hours	System database and persistent storage engine	Restarts services, restablishes network connections, and recovers runtime application state	System maintains more than 99% uptime during business hours with 0 confirmed reservations or submitted orders lost (100% retention across 5 restarts)	QR-03, QR-05

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
Layered with event driven messaging	QAS-01, QAS-02, QAS-03, CON-02 Provides low-latency async order dispatching to the KIDS via web sockets while isolating data access for Acid crash	Increases deployment complexity by requiring local message broker/event bus alongside application tiers, and handling reconnections.	Chosen
Microservice architecture	QAS-02 Independent horizontal scaling of reservation and order processing across separate containers	Significant distributed system overhead, eventual consistency hazards, complex network orchestration, and etc.	Rejected
Shared repository	QAS-03 Centralizes all state persistence directly in a single shared database instance	Heavy polling bottlenecks that cause high database lock contention, and can go beyond 5 second latency which fails QAS-01	Rejected

A Layered 3-tier architecture complemented by event driven messaging is the most optimal because it addresses the throughput and fault tolerant quality constraints established before. Seperating the layer, and business logic layer, and persistence layer ensures that high volume customer browsing, QAS-02, and QAS-03 works as the former is isolated from database locks and there is zero data loss in unexpected restarts. Having event driven socket channels between server terminal and kitchen display bypasses database polling latency, and ensure sub 5 second latency in a local network. As mentioned before microservices was rejected due to unnecessary complexity and a shared database repository style will not be able to handle proper speeds with 20+ tables created.

6.3 Component view

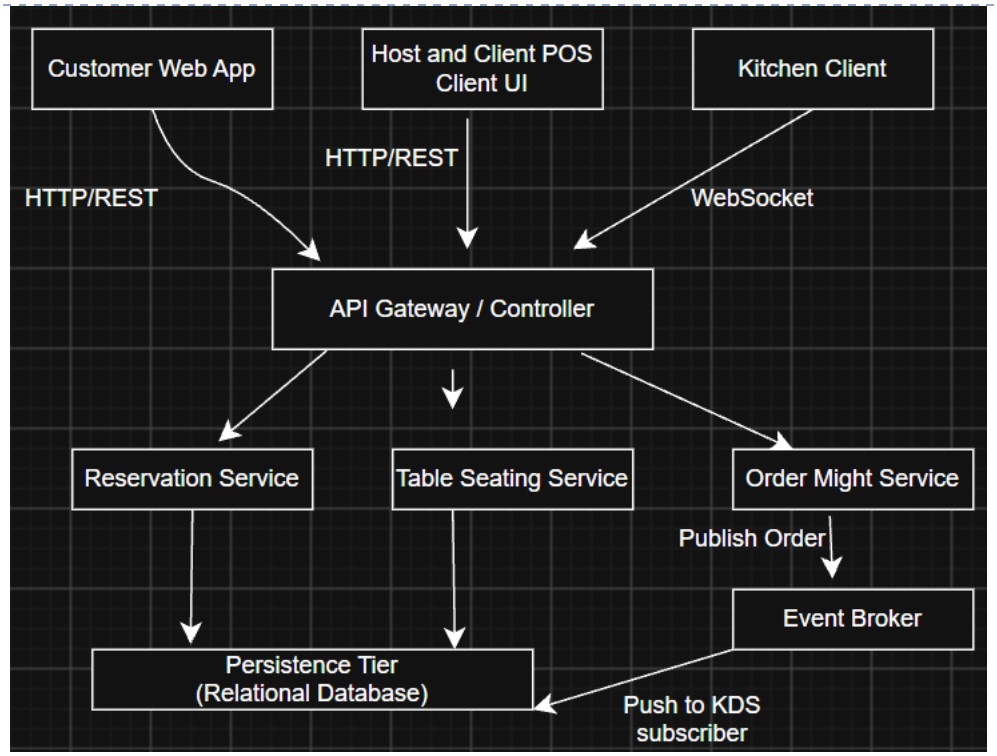


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Customer Web App	Services the browser-based client interface for customers to check availability and submit table bookings	HTTP/UI rendering Form input events	IReserverationAPI	UC-01, QAS-02
Host and Server POS UI	Delivers the local staff interface for managing floor layouts, assigning party seating, and entering table orders	Staff touch UI, Floor plan view, Order input controls	IReservationAPI, ITableSeatingAPI, IOrderAPI	UC-01, UC-04, UC-05
Kitchen Display UI	Renders incoming kitchen tickets in real time and captures line-item status updates	Touch-target ticket interface	IKitchenQueueAPI, IEventSubscriber	QR-04, UC-05, QAS-01
API Gateway/ Controller Tier	Authenticates client requests, enforces request validation, and routes traffic to domaiN.	IReservationAPI, ITableSeatingAPI, IOrderAPI, IKitchenQueueAPI	IReservationService, ISeatingService, IOrderService	QAS-02

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Reservation Service	Evaluates table slot availability, checks party size capacities, prevents overlapping bookings, and manages reservation state.	IReservation Service	IDatabaseAccess	UC-01, QAS-02, QAS-03, ADR-01
Table Seating Service	Manages physical table states, associates walk-in and arriving parties to tables, and tracks active seating sessions.	ISeatingService	IDatabaseAccess	UC-04, QAS-03,
Order Service	Validates item availability against menu catalog, persists customer orders, and emits order placement events.	IOrderService	IDatabaseAccess, IEventEmitter	UC-05, QAS-01, QAS-03
Event Brker	Handles asynch socket messaging across local network to push new events instantly	IEventEmitter, IEventSubscriber	Local WebSocket / TCP socket transport	QAS-01
Persistence/ Data Tier	Manages ACID transactions, structured queries, and durable storage for tables, reservations, sessions, and orders.	IDatabaseAccess	Underlying filesystem storage engine	QAS-03

6.4 Deployment view

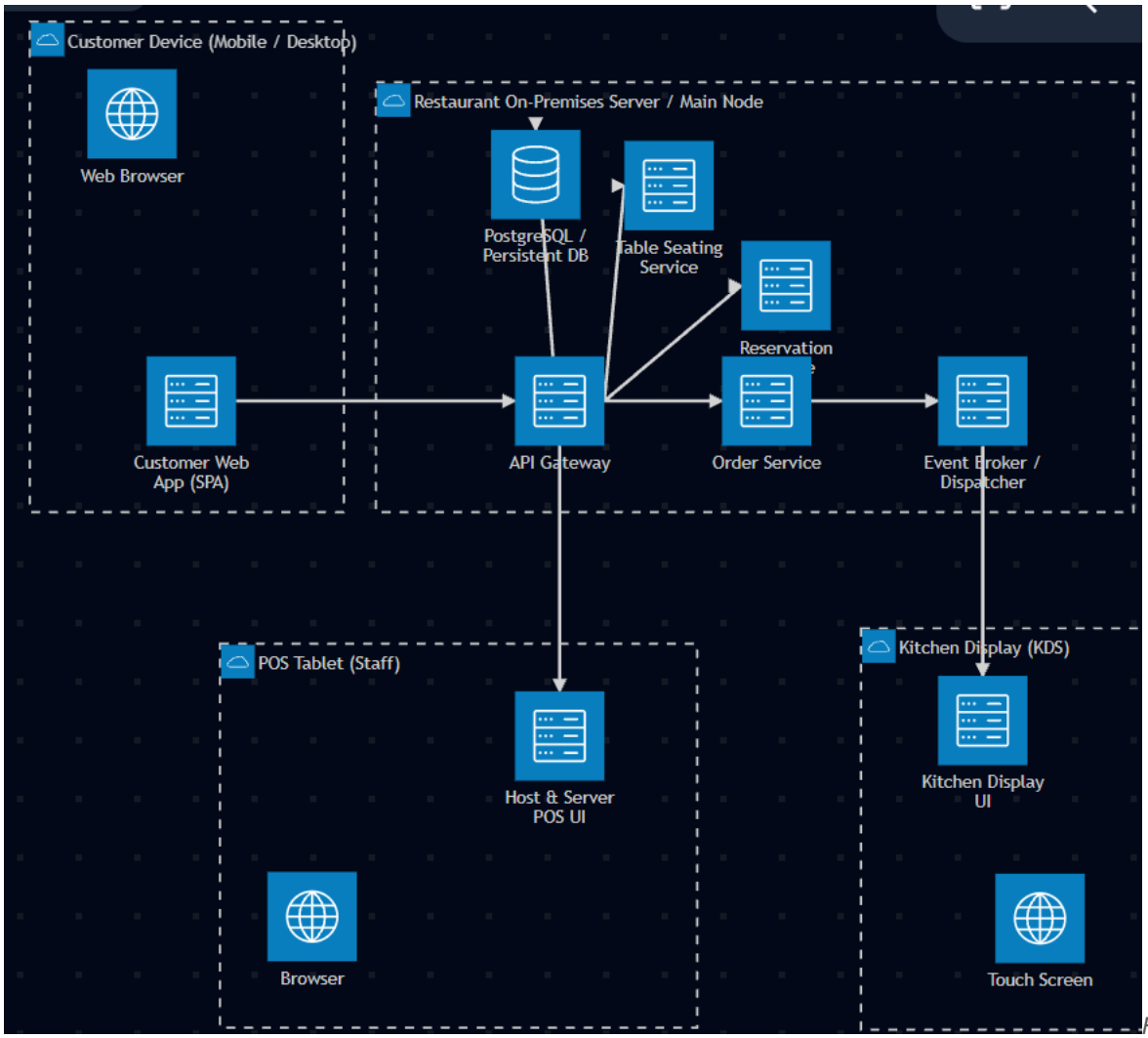


Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
Customer Mobile / PC	Customer Web UI Client (compiled SPA assets, browser runtime)	Reverse Proxy via HTTPS (Port 443) over public internet	Untrusted (Public)
Reverse Proxy / Edge Gateway	Reverse proxy server (e.g., NGINX / Caddy), SSL/TLS termination, rate limiting	Customer Web UI via HTTPS; Application Server via Internal HTTP	DMZ (Perimeter)

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
On-Premises Application Server	Reservation Service, Order Service, Kitchen Dispatch Engine (containerized micro-runtimes)	Reverse Proxy via HTTP; POS Tablet via HTTPS (LAN); KDS Unit via WebSocket Database Server via SQL/TCP	Trusted (Internal Local Subnet)
Database Server Node	PostgreSQL	Reservation Service and Order Service via SQL over TCP (Port 5432 / Loopback)	Trusted (Internal Secure Store)
Server POS Tablet	POS / Server UI Client (Staff management and order entry app)	Application Server (Order Service, Reservation Service) via HTTPS over WPA3 Enterprise Wi-Fi	Trusted (Internal Restaurant LAN)
Kitchen KDS Unit	KDS Display UI Client (Real-time kitchen order queue interface)	Application Server (Kitchen Dispatch Engine) via WSS (Secure WebSockets) over LAN Wi-Fi / Ethernet	Trusted (Internal Restaurant LAN)

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
ReservationUI / ReservationController	Customer Web App / Reservation Service	Boundary UI and declared interaction controller for UC-01 (5.2.1)
OrderPlacement	OrderService	Declared interaction controller for UC-05 (5.2.2)
KitchenQueue	Order Service / Event Broker / Kitchen Display UI	Merged: represents active Order records (status = queued) pushed via Event Broker to Kitchen Display UI
MenuItem	Menu Service	Maintains Menu details and item availability
OrderLine (OrderItem)	Order Service / Persistence Tier	Renamed: OrderLine in UC-05 (5.2.2) maps directly to the OrderItem domain entity in Section 5.1

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
Order	Order Service	Groups menu items, quanties, and preparation notes
Server	Order Service	Supports table-linked order submission
Host	Seating Service	Supports table assignment and release
Table	Seating Service	Maintains table capacity, the occupancy, and the availaibilty,
Reservation	Reservation Service	This handles the booking, confirmation, and reservation changes,
Customer	Reservation Service	Maintains customer details needed for this booking
Restaurant	Restaurant Configuration Service	This manages restaurant hours and reservation rules

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	Reservation booking, seating, order management, and kitchen queue processing have separate responsibilities	Reservation Service, Seating Service, Order Service, Kitchen Queue Service
Information hiding	The customer and staff interfaces access reservation and order information through the responsible services rather than basically directly modifying stored records.	Reservation Service, Order Service
Low coupling	Frontended interfaces basically communicate with backend services through defined interfaces, allowing components to change independently	Reservation Interface, Order Interface
High cohesion	Each component focuses on one related set of tasks such as managing reservations or processing orders	Reservation Service, Order Service

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Design for change	The menu availability and reservation rules can be updated within their responsible components without changing unrelated feautures	Menu Service, Restaurant Config service,
Keep it simple (KISS)	This system basically uses a straightforward webased architecture with a React frontend and a Node.js backend, and a persistent storage rather than an unnecessary distributed services	Node.js application server, react frontend

7 Design Rationale

7.1 Architecture decision records

ADR-01 Layered three-tier architecture with event-driven messaging

Status	Accepted
Date and owner	Nov 7, 2026 · Jarir Ali
Context (drivers)	QAS-01 (order on the KDS within 5 s), QAS-02 (customer pages within 3 s for 100 concurrent users), QAS-03 (no confirmed reservation or order lost across a restart) and CON-02 (touch-only KDS).
Options considered	(1) Layered three tiers (presentation, business logic, persistence) with event-driven messaging to the KDS. (2) Microservices, scaling reservation and order processing independently. (3) Shared repository, with every client reading state from one database.
Decision	Option 1, as analysed in section 6.2.
Consequences	Favors performance and reliability (QAS-01, QAS-02, QAS-03). Move: separate the presentation, business and data tiers, and push kitchen events instead of reading the database. Cost: an event broker must be deployed alongside the tiers, and clients must handle reconnections. Microservices were rejected for their distributed-system overhead; the shared repository for polling latency and lock contention.
Evidence	Spike S-01 (section 8): event push met QAS-01 on degraded Wi-Fi, while polling, the shared repository's access pattern, delivered only 82% of orders within 5 s.
Related	ADR-02, ADR-03, QAS-01, QAS-02, QAS-03, R6

ADR-02 Push kitchen orders to screens over WebSocket instead of polling

Status	Accepted
Date and owner	Nov 7, 2026 · Manideep Pendyala
Context (drivers)	QAS-01 (order on the KDS within 5 s for 95% of orders), UC-05 with FR-09 and FR-11, and A1 (restaurant Wi-Fi is the only link between front and back of house).
Options considered	(1) KDS and server screens poll the order service every 3 s. (2) The order service pushes each new order and status change over a persistent WebSocket connection.
Decision	Option 2, WebSocket push. After a reconnect, a screen requests the orders it missed.
Consequences	Favors performance (QAS-01). Move: replace polling with server push. Cost: the order service must manage open connections and reconnects, and a screen that drops off the Wi-Fi depends on its catch-up request (R6).
Evidence	Spike S-01 (section 8): on degraded Wi-Fi, push delivered 100% of orders within 5 s; polling delivered 82%.

Related	ADR-03, QAS-01, R6

ADR-03 Save every order to the database before notifying any screen

Status	Accepted
Date and owner	Nov 7, 2026 · Manideep Pendyala
Context (drivers)	QAS-03 (all confirmed reservations and submitted orders survive a restart), FR-03 (no overlapping table allocation), and the UC-05 minimal guarantee (a failed submission leaves no kitchen entry).
Options considered	(1) Keep the kitchen queue in the order service's memory and save it to the database in the background. (2) Save each order in a PostgreSQL transaction first, then notify screens.
Decision	Option 2. PostgreSQL is the single source of truth; screens are notified only after the save commits.
Consequences	Favors reliability (QAS-03). Move: commit before notify. Cost: a database write is added to the QAS-01 path, and the database becomes a single point of failure (R7).
Evidence	None yet. S-01 kept orders in memory; open question Q5.
Related	ADR-02, QAS-01, QAS-03, R7

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Performance and reliability (QAS-01, QAS-02, QAS-03)	Layered tiers with event-driven messaging instead of microservices or a shared repository	An event broker to deploy and run; clients handle reconnections	ADR-01
Performance (QAS-01)	Server push over WebSocket instead of polling	Connection and reconnect handling on the order service	ADR-02
Reliability (QAS-03)	Commit to the database before notifying any screen	Extra latency on the order path; the database is a single point of failure	ADR-03

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01 Push vs polling for kitchen order delivery
Risk or decision it addresses	R6 / ADR-02 / QAS-01
Question (answerable yes or no)	Does pushing orders over a WebSocket put 95% of 50 orders from 20 tables on the KDS within 5 s, and does polling every 3 s also do so?
Set-up	Node.js 24 with the ws library; orders kept in memory. One script runs the order service, a push KDS, a polling KDS and 20 simulated tables submitting 50 orders over 30 s. Wi-Fi is simulated as 5–80 ms per hop; normal Wi-Fi stalls 2% of hops by 1 s, degraded Wi-Fi stalls 20% of hops by 2 s. Folder: /spikes/S-01/.
How it was measured	For each order, KDS receive time minus submit time on one clock; p50, p95 and the share within 5 s.
Result (numbers)	Normal Wi-Fi: push p95 138 ms, polling p95 2997 ms; both 100% within 5 s. Degraded Wi-Fi: push p95 4082 ms, 100% within 5 s; polling p95 6188 ms, 82% within 5 s.
What it changed or confirmed	Confirmed ADR-02: only push meets QAS-01 on degraded Wi-Fi.
Where to see it	Commit 30be6d7; /spikes/S-01/ (README.md, results-nominal.json, results-degraded-wifi.json)

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01 (MUST)	UC-01 Reserve Table	Customer, Reservation, Table, ReservationController	Customer Web App, API Gateway, Reservation Service, Persistence Tier	QAS-02, ADR-01	Sec 4.2 UC-01 step (2), Fig 5.2.1, FR-01 slot search acceptance test
FR-02 (MUST)	UC-01 Reserve Table	Customer, Reservation, Table, Notification, ReservationController	Customer Web App, API Gateway, Reservation Service, Persistence Tier	QAS-02, QAS-03, ADR-01, ADR-03	Sec 4.2 UC-01 step (6), Fig 5.2.1, unique confirmation number test
FR-03 (MUST)	UC-01 Reserve Table	Reservation, Table, ReservationController	Reservation Service, Persistence Tier	QAS-03, ADR-01, ADR-03	Sec 4.2 UC-01 step (2)/A1, Fig 5.1 constraint note, overlap rejection test
FR-04 (SHOULD)	UC-02 Modify Reservation	Customer, Reservation, Table	Customer Web App, Reservation Service, Persistence Tier	QAS-02, QAS-03, ADR-01	Sec 3.1 FR-04 acceptance criterion (deferred to P3 per MoSCoW)
FR-05 (SHOULD)	UC-03 Cancel Reservation	Customer, Reservation, Table	Customer Web App, Reservation Service, Persistence Tier	QAS-02, QAS-03, ADR-01	Sec 3.1 FR-05 acceptance criterion (deferred to P3 per MoSCoW)
FR-06 (MUST)	UC-04 Manage Table Seating	Staff, Reservation, Table, SeatingSession, WaitlistEntry	Host and Server POS UI, API Gateway, Table Seating Service, Persistence Tier	QAS-03, ADR-01, ADR-03	Sec 4.2 UC-04 steps (1)–(6), Sec 3.1 FR-06 seating assignment test
FR-07 (MUST)	UC-05 Place Table Order	Staff, Table, SeatingSession, Order, OrderItem (OrderLine), MenuItem, OrderPlacement	Host and Server POS UI, API Gateway, Order Service, Persistence Tier	QAS-01, QAS-03, ADR-01,	Sec 4.2 UC-05 steps (1)–(5), Fig 5.2.2 M01–M18, table-linked order test

				ADR-03	
FR-08 (SHOULD)	UC-05 Place Table Order	Order, OrderItem (OrderLine), MenuItem, OrderPlacement	Host and Server POS UI, Order Service, Persistence Tier	ADR-01, ADR-03	Sec 4.2 UC-05 A3/A5, Fig 5.1 MenuItem note, Fig 5.2.2 M15–M16
FR-09 (MUST)	UC-05 Place Table Order	Order, OrderItem (OrderLine), KitchenQueue, OrderPlacement	Order Service, Event Broker, Kitchen Display UI, Persistence Tier	QAS-01, QAS-03, ADR-01, ADR-02, ADR-03	Sec 4.2 UC-05 step (6), Fig 5.2.2 M17–M21, Spike S-01 (commit 30be6d7)
FR-10 (MUST)	UC-07 Prepare Order	Staff, Order, OrderItem, KitchenQueue	Kitchen Display UI, API Gateway, Order Service, Event Broker, Persistence Tier	QAS-01, QAS-03, ADR-01, ADR-02, ADR-03	Sec 3.1 FR-10 status transition (queued -> preparing -> ready) test
FR-11 (MUST)	UC-06 Track and Serve Order	Staff, Order, SeatingSession, Notification	Kitchen Display UI, Event Broker, Order Service, Host and Server POS UI	QAS-01, ADR-01, ADR-02	Sec 3.1 FR-11 server order view status update test, Spike S-01
FR-12 (SHOULD)	UC-06 Track and Serve Order	Staff, Order, SeatingSession, KitchenQueue	Host and Server POS UI, Order Service, Event Broker, Persistence Tier	QAS-03, ADR-01, ADR-03	Sec 3.1 FR-12 served status & active queue removal test
FR-13 (MUST)	UC-04 Manage Table Seating	Staff, Table, SeatingSession, Reservation	Host and Server POS UI, API Gateway, Table Seating Service, Persistence Tier	QAS-03, ADR-01, ADR-03	Sec 4.2 UC-04 steps (7)–(10), table release & future reservation check
FR-14 (COULD)	UC-08 Manage Menu Availability	Staff, MenuItem	Host and Server POS UI, API Gateway, Order Service, Persistence Tier	ADR-01, ADR-03	Sec 3.1 FR-14 menu item availability toggle test

QR-01 (MUST)	UC-05, UC-06, UC-07	Order, OrderItem, KitchenQueue, OrderPlacement	Order Service, Event Broker, Kitchen Display UI	QAS-01, ADR-01, ADR-02	Spike S-01 (30be6d7): WebSocket push p95 = 4082 ms (100% <= 5 s)
QR-02 (MUST)	UC-01, UC-02, UC-03	Customer, Reservation, Table, ReservationController	Customer Web App, Reverse Proxy, API Gateway, Reservation Service	QAS-02, ADR-01	Sec 3.2 & 6.1 100-concurrent-user load test (< 3 s for 95% of requests)
QR-03 (MUST)	UC-01 to UC-08	Reservation, SeatingSession, Order, Table	On-Premises Application Server, Persistence Tier (PostgreSQL)	QAS-03, ADR-01, ADR-03	Sec 3.2 & 6.1 multi-day operating hours uptime monitoring (>= 99%)
QR-04 (SHOULD)	UC-07 Prepare Order	Staff, Order, OrderItem, KitchenQueue	Kitchen Display UI	CON-02, ADR-01	Sec 3.2 KDS UI walkthrough (<= 2 taps, >= 44x44 px touch targets)
QR-05 (MUST)	UC-01, UC-04, UC-05, UC-07	Reservation, SeatingSession, Order, OrderItem	Reservation Service, Table Seating Service, Order Service, Persistence Tier	QAS-03, ADR-01, ADR-03	Sec 3.2 & 6.1 5-restart persistence test (40/40 records retained)
CON-01	UC-01, UC-02, UC-03	Customer, ReservationUI	Customer Web App (SPA)	ADR-01	Cross-browser check on latest 2 versions of Chrome, Safari, Firefox, Edge
CON-02	UC-07 Prepare Order	Staff, KitchenQueue	Kitchen Display UI (Kitchen KDS Unit)	QAS-01, ADR-01	Touch-only inspection on >= 10-inch KDS screen without keyboard/mouse

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	None (0 items). Earlier draft had FR-14 (MenuItem availability) pointing to a separate Menu Service in Section 6.5 that was missing from Figure 6.3 and Table 6.3.	Consolidated MenuItem catalog and availability checks into Order Service and Persistence Tier in Sections 6.3, 6.5, and 9.1 so every functional (FR-01–FR-14), quality (QR-01–QR-05), and constraint (CON-01–CON-02) requirement maps to an explicit component in Table 6.3.
Components with no requirement, QAS or ADR	None (0 items). Every component in Table 6.3 and node in Table 6.4 traces to at least one requirement, QAS, or ADR.	Verified that infrastructure tiers (API Gateway / Controller Tier, Event Broker, Persistence / Data Tier, and Reverse Proxy) trace directly to QAS-01, QAS-02, QAS-03, and ADR-01–ADR-03.
Use cases not analyzed (and why that is acceptable)	UC-02 (Modify Reservation), UC-03 (Cancel Reservation), UC-06 (Track and Serve Order), UC-07 (Prepare Order), and UC-08 (Manage Menu Availability).	Acceptable for P2 because the 3 expanded use cases (UC-01, UC-04, UC-05) exercise all core domain lifecycles (Reservation, SeatingSession, Table, Order, OrderItem, MenuItem, KitchenQueue) and all 3 architectural pathways (REST booking, host seating, and WebSocket KDS push). UC-02, UC-03, and UC-08 realize SHOULD/COULD requirements (FR-04, FR-05, FR-14), while UC-06 and UC-07 are simple state-transition updates (queued -> preparing -> ready -> served) over the exact same Order Service, Event Broker, and Kitchen Display UI pipeline already modeled in UC-05.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

Roles and decision rules are unchanged since P1, except that Manideep Pendyala (Usability and Prototype Evidence Lead) also owns spike S-01, ADR-02 and ADR-03.

10.2 Plan to P3

Milestone	Date	Owner	Done when
Application skeleton on main	Nov 18	Jarir Ali	The front end, application services and database schema build from a clean clone.
Reservation and seating (FR-01 to FR-06, FR-13)	Nov 24	Mehak Jay, Vasudha Silawat	UC-01 and UC-04 acceptance criteria pass.
Ordering and kitchen (FR-07 to FR-12, FR-14)	Nov 28	Gabriel Suarez	UC-05 acceptance criteria pass.
S-01 rerun on the integrated build	Dec 1	Manideep Pendyala	QAS-01 measured with the database write in place (closes Q5).
Traceability extended to code and tests	Dec 5	Arpita Lamichane	Every MUST requirement maps to a test.
P3 submitted	Dec 9	Sandip Nepal	PDF tagged p3 and uploaded to Canvas.

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1 Important actors missing from scope	Downgraded: stakeholder needs now rest on interview notes in /docs/elicitation/interviews.md.	Low / High	Recheck actors when P3 adds features. Owner: Sandip Nepal.
R2 Prototype evidence inadequate	Closed: spike S-01 is committed with measured results.	—	—
R3 Team disagreement stalls decisions	Open, unchanged.	Low / Medium	Record each settled design debate as an ADR. Owner: Sandip Nepal.

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R4 Traceability falls out of sync	Open.	Medium / Medium	Update the 9.1 matrix in the same pull request as any design change. Owner: Arpita Lamichane.
R5 Members fall behind on their sections	Open.	High / Medium	Reassign any section not started 48 hours before a deadline. Owner: Sandip Nepal.
R6 (new) A Wi-Fi drop disconnects the KDS and orders go unseen	Created by ADR-02.	Medium / High	Automatic reconnect plus a catch-up request (ADR-02); measured in S-01. Owner: Gabriel Suarez.
R7 (new) A database outage stops reservations and orders	Created by ADR-03.	Low / High	Run the QAS-03 restart test at every milestone. Owner: Jarir Ali.

10.4 Repository

The project repository is publicly hosted and accessible at https://github.com/sandip10nepal/cs3354-project-p1. A root README.md file documents the team roster, directory layout, branching rules, and communication schedule. The repository is organized into the following directory structure:

- **/docs/p1/:** Milestone P1 baseline artifacts (v0.8).
- **/docs/p2/:** The compiled Milestone P2 baseline report PDF (CS3354_P2_Report.pdf) and individual section Markdown drafts under /docs/p2/sections/.
- **/docs/p2/diagrams/:** Editable source files (.drawio and PlantUML) for the use-case diagram, conceptual domain model, analysis sequence diagrams (UC-01 and UC-05), component view, and UML deployment view.
- **/docs/elicitation/interviews.md:** Stakeholder interview transcripts and workflow observation notes referenced as [R3].
- **/spikes/S-01/:** Source code, benchmark output logs (results-nominal.json, results-degraded-wifi.json), and README.md for Architecture Spike S-01.

Branch, Pull-Request, and Communication Rules: All team members develop their assigned sections, diagram sources, and spike code on dedicated feature branches (feature/-) using their own GitHub accounts. Changes are merged into main exclusively through Pull Requests that have been reviewed and approved by at least one teammate. The team coordinates through twice-weekly synchronous check-ins on Microsoft Teams/Discord (Tuesdays and Fridays at 7:00 PM CST) and daily asynchronous status updates in the group chat. The Git tag p2 points directly to the commit on main that contains the exact final PDF submitted to Canvas.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
Manideep Pendyala	Claude Code	Under my direction: wrote the spike S-01 code and tests, and drafted sections 7, 8, 10.1–10.3 and 11 from the scope, decisions and format I set (ADR-01 deferred to 6.2, P3 dates, template format, content limits).	Reviewed all spike code; ran the tests and both benchmark runs; cut and reformatted the drafts into the template; checked every number against the result files and every ID against sections 3, 4.2 and 6.1.
Sandip Nepal	Gemini	I used Gemini to divide project sections to team members.	I analysed the P2 template using Gemini to summarise.
Jarir Ali	Gemini	Gemini and for research for optimal architecture design and drafting	Drafting rough drafts

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	Does prototype evidence require a working prototype?	Manideep Pendyala	Instructor, Oct 20	Closed: the P2 brief defines it as a measured architecture spike (S-01).	—
Q2	Do customers need user accounts, or can they book as guests?	Manideep Pendyala	Sandip Nepal, Nov 18	Open.	Customers book as guests and manage a booking by its confirmation number (FR-02); only staff have accounts.
Q3	Which languages and frameworks will the project use?	Manideep Pendyala	Jarir Ali, Nov 8	Closed: section 6.2 and ADR-01 set a layered three-tier style with event-driven messaging; section 6.4 places the data on PostgreSQL.	—

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q4	Does WebSocket push meet QAS-01 on real restaurant Wi-Fi?	Manideep Pendyala	Manideep Pendyala, Dec 1	Open: S-01 simulated the Wi-Fi.	The simulated degraded profile is no better than a busy restaurant.
Q5	Does saving to the database before notifying the KDS keep QAS-01 within 5 s?	Manideep Pendyala	Manideep Pendyala, Dec 1	Open: S-01 kept orders in memory.	A single-order insert adds well under 1 s.

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date
Sandip Nepal	S.N	Nov 7, 2026
Arpita Lamichane	A.L	Nov 7, 2026
Mehak Jay	M.J	Nov 7, 2026
Vasudha Silawat	V.S	Nov 7, 2026
Gabriel Suarez	G.S	Nov 7, 2026
Jarir Ali	J.A	Nov 7, 2026
Manideep Pendyala	M.P	Nov 7, 2026